

Chapter 13 — Access

Keys, endpoints, and what a call costs

The cost thread has run through six chapters as a physical quantity. Here it becomes an invoice.



What a key authorises

- A key is a bearer credential. Whoever holds it is you, to the billing system.
- A paid chat subscription does not include API access. They are separate products with separate billing, and this surprises people every time.
- API usage is billed against prepaid credit, per token. **There is no monthly subscription cap** — your exposure is the credit you have loaded, plus whatever auto-reload you switch on. Prepaid is the ceiling; auto-reload is how people remove it without noticing.



Key hygiene, in four rules

- Environment variables, never source. A key in a script is a key in your shell history and your editor's autosave.
- Never in a shared repository. Not in a private one either — private repositories get shared.
- One key per person, so a leak can be traced and revoked without stopping everyone.
- Rotate on any doubt. Rotation is cheap. Establishing that a key was not misused is not.

The group's local gateway issues per-user keys with per-user budgets. That is the mechanism that makes an exercise safe to run in a room.



Worked example: architecture from a price list

A published price step at a stated context length is enough to solve for something the vendor never published — if you state every input.

C_{extra} [cost] extra charge on one request, attributable to holding the long context

L [token] context length the step is quoted at

t_{res} [hour] how long the cache stays resident while the request is served

p_{bh} [cost · byte⁻¹ hour⁻¹] price of one byte-hour of device memory

$$b_{\text{kv}} = \frac{C_{\text{extra}}}{L t_{\text{res}} p_{\text{bh}}}$$
$$\frac{\text{cost}}{\text{token} \cdot \text{hour} \cdot \text{cost} \cdot \text{byte}^{-1} \text{hour}^{-1}} = \frac{\text{byte}}{\text{token}} \quad \checkmark$$



The two inputs that decide the answer

Neither is published. Both have to be constructed, and the result is only as good as they are.

- **Residency time** t_{res} . The cache occupies memory for the duration of the request, not for an hour. Get this wrong by 10× and the answer is wrong by 10×.
- **The byte-hour price** p_{bh} . No vendor quotes one. You build it: accelerator price per hour ÷ bytes of memory per accelerator.

This is the denominator problem again. The house rule from Chapter 4 says that where an equation's denominator is defined non-obviously, the definition goes beside it. p_{bh} is exactly that, and it is the first thing a hostile reader attacks. State it on the slide or do not put the number up.



Sanity-check it, and watch the check strain

The figure recorded from this method is roughly **1.7 kB per token**. Test it against architecture:

$$b_{\text{kv}} = \underbrace{2}_{K \text{ and } V} \times n_{\text{layers}} \times n_{\text{kv}} \times d_{\text{head}} \times \underbrace{2}_{\text{byte}}$$

- Setting that to 1700 bytes needs $n_{\text{layers}} \times n_{\text{kv}} \times d_{\text{head}} \approx 425$.
- At 60 layers that is about **7 per layer** — smaller than a single head of dimension 64.
- So either the model compresses KV far past ordinary grouped-query attention, or the **1.7 kB** is per layer, not whole-model.

TODO(verify) — the deck does not currently know which. Read the source transcript and state it. An undeclared per-what is the MACs-versus-FLOPs erratum wearing a different hat, three chapters after that erratum was taught as the anchor of Chapter 4.

And re-run the arithmetic on current prices before delivery. The method transfers; the number does not.



Why that example is in this course

- Order-of-magnitude estimation from public numbers, which is a skill this room already has and does not apply here.
- Dimensional analysis — the units carry the derivation, exactly as in Chapter 7.
- An independent check. The answer is only worth anything because it was tested against something it was not derived from.

The whole point is that you can do this. A price list is public, an architecture is not, and dimensional analysis crosses the gap — provided every input is declared. Two slides ago one of them was not, which is why the check strains.



Your first call

The brief for this session is not "understand the API". It is **make one call and change one thing**.

1. Point a client at the group's gateway with your own key. Confirm you get a response at all before changing anything.
2. Send the same prompt twice, unchanged. **You already know what to expect** — Chapter 1 said the sampler is stochastic and Chapter 4 made you measure the spread.
3. Now vary one parameter at a time — temperature, max tokens, the system prompt — and watch which ones change the answer and which change only the bill.
4. Read the usage figures that come back with the response. That is the number Chapter 7 derived and this chapter prices.

BLOCKED — needs the local machine reachable from the training room. `DECISIONS.md` item 4 blocks *this* exercise, not only the comparison demo two slides on. If the machine is not reachable, this becomes an instructor demo and keys go out afterwards.



Local against frontier, side by side

- Same prompt. The group's own local model, and a frontier model. Both projected.
- This is the best available demonstration of everything in Chapter 5, on hardware the group already owns.
- Expect the local model to be worse in a specific, visible, instructive way — not uniformly worse.

TODO(capture) — needs the local machine reachable from the training room. If it is not, this becomes an instructor demo and keys are distributed afterwards. DECISIONS.md item 4.



Closing the cost thread

- **Metered pricing.** Per token, in and out, at different rates. Chapter 1's unit of meaning, priced.
- **Caching.** Re-sending the same standing context every turn is the cost Chapter 10 made you calculate. Caching is the mitigation, and it has its own price.
- **Batch discounts.** Latency traded for money, which is Chapter 7's roofline seen from the finance side.
- **Subscription against per-token.** Different economics, and the crossover depends on how you actually work, not on the headline rate.

Thread — cost, closed. Token as unit of meaning in Chapter 1, unit of price in Chapter 7, resident context in Chapter 10, orchestration premium in Chapter 11, and an invoice here.



Where this leaves us

- A credential model you can explain to whoever administers your grant.
- A worked example where public prices yielded an unpublished architectural number.
- A cost thread that started as a token and finished as a line item.

Session 2 ends here. Session 3 is your actual work.

